PROGRAMMING 512 ASSIGNMENT

RICHFIELD
richfield.ac.za

Name and Surname: Ashley Sithole

Student ITS No: 402418437

Qualification: Bscl.T **Year of Study:** 1 **Semester:** 2

Assignment due date: 28 October 2026 **Date submitted:** 28 October 2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document. Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents

Introduction

Main Body (with relevant subheadings)

Conclusion

References

Submit the assignment in PDF format on Moodle. Use the specified cover page provided.

Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism

Student Signature: SMAMS **Date:** 28 October 2026

Table of Contents

Introduction 3
Git Hub Link 3
Main.py code 4
Models.py codes 19
Database.py codes 23
Readme.md 29
Conclusion 30
References 31

Introduction

Richfield Graduate Institution operates across several campuses in South Africa and depends on shared academic resources such as computer laboratories, multimedia projectors and seminar rooms. At present, many campuses manage bookings manually through emails, spreadsheets and paper registers. This approach leads to double bookings, underused resources, a lack of accountability and difficulty in producing accurate usage reports. Management also has no single view of how resources are used across campuses.

This report presents the design and implementation of a Smart Campus Resource Booking System developed in Python to address these problems. The system uses Tkinter for the graphical user interface and an SQLite database to store data permanently between program runs. It supports three user roles, namely lecturers, campus administrators and system operators. These roles are implemented with object-oriented programming concepts such as classes, inheritance and encapsulation. Each of the three campuses (Johannesburg, Durban and Cape Town) applies its own booking policy, covering maximum booking duration and permitted booking hours. These policies are kept separate from the core logic so that new campuses or rules can be added easily.

The report explains the system design, the main features, the database structure and how the booking rules are enforced. It also describes how the system generates campus-level and cross-campus reports. It ends with a conclusion on what was achieved and what could be improved.

Git Hub Link

https://github.com/kingmonty202-gif/Programming-512-Assignment-final.git

Main.py code

```
"""main.py - Tkinter GUI for the Smart Campus Resource Booking System.
Run with:  python main.py
"""
import tkinter as tk
from tkinter import ttk, messagebox
from datetime import datetime

import database as db
import models


# ---------- colours (change these to restyle the whole system) ----------
BG_COLOR = "#ADD8E6"          # light blue window background
BUTTON_COLOR = "#2B6CB0"      # button background
BUTTON_HOVER = "#1E4E8C"      # button colour when the mouse is over it
ROLE_COLORS = {               # top bar colour for each user role
    "lecturer": "#1F6FB2",    # blue
    "admin": "#FFA500",       # orange
    "operator": "#ff0000",    # red
}


# ---------- small helper functions for tables ----------
def make_tree(parent, columns, widths):
    tree = ttk.Treeview(parent, columns=columns, show="headings", height=12)
    for col, width in zip(columns, widths):
        tree.heading(col, text=col)
        tree.column(col, width=width)
    tree.pack(fill="both", expand=True, padx=5, pady=5)
    return tree


def fill_tree(tree, rows):
    """First value of each row is the database id (used as the row id)."""
    tree.delete(*tree.get_children())
    for row in rows:
        tree.insert("", "end", iid=str(row[0]), values=row[1:])


def booking_rows(rows):
    """Change the raw status into Upcoming / Past / Cancelled and format times."""
    result = []
    for b in rows:
        state = models.booking_state(b[4], b[5], b[7])
        result.append((b[0], b[1], b[2], b[3], b[4], f"{b[5]:02d}:00", f"{b[6]} h", state))
    return result
```

```
BOOKING_COLS = ("Resource", "Campus", "Lecturer", "Date", "Start", "Duration", "Status")
BOOKING_WIDTHS = (160, 110, 100, 90, 60, 70, 90)


class App:
    def __init__(self, root):
        self.root = root
        root.title("Smart Campus Resource Booking System")
        root.geometry("900x600")
        root.configure(bg=BG_COLOR)
        self.style = ttk.Style()
        self.setup_colors()
        self.user = None
        self.campus = None          # (id, name) of the active campus
        self.refreshers = []        # functions that reload the tables
        self.show_login()

    def setup_colors(self, accent="#1F6FB2"):
        """Apply background and button colours to all ttk widgets."""
        st = self.style
        st.theme_use("clam")      # 'clam' lets us change colours on every platform
        st.configure("TFrame", background=BG_COLOR)
        st.configure("TLabel", background=BG_COLOR)
        st.configure("TNotebook", background=BG_COLOR)
        st.configure("TNotebook.Tab", padding=(12, 5))
        st.configure("TButton", background=BUTTON_COLOR, foreground="white", padding=5)
        st.map("TButton", background=[("active", BUTTON_HOVER)])
        st.configure("Treeview", rowheight=24)
        st.configure("Treeview.Heading", background=accent, foreground="white",
                     font=("Arial", 10, "bold"))
        # The top bar (campus / user info) uses the role colour
        st.configure("Top.TFrame", background=accent)
        st.configure("Top.TLabel", background=accent, foreground="white")

    def clear(self):
        for widget in self.root.winfo_children():
            widget.destroy()
        self.refreshers = []

    # ================== LOGIN / REGISTER ==================
    def show_login(self):
        self.clear()
        self.user = None
        frame = ttk.Frame(self.root, padding=30)
        frame.pack(expand=True)
        ttk.Label(frame, text="Smart Campus Resource Booking System",
                  font=("Arial", 16, "bold")).grid(row=0, column=0, columnspan=2, pady=15)
        ttk.Label(frame, text="Username:").grid(row=1, column=0, sticky="e", pady=5)
        ttk.Label(frame, text="Password:").grid(row=2, column=0, sticky="e", pady=5)
        self.username_entry = ttk.Entry(frame, width=25)
        self.password_entry = ttk.Entry(frame, width=25, show="*")
```

```
        self.username_entry.grid(row=1, column=1, pady=5)
        self.password_entry.grid(row=2, column=1, pady=5)
        ttk.Button(frame, text="Login", command=self.login).grid(row=3, column=0, columnspan=2,
pady=8)
        ttk.Button(frame, text="Register as Lecturer", command=self.register).grid(row=4, column=0,
columnspan=2)

    def login(self):
        username = self.username_entry.get().strip()
        password = self.password_entry.get()
        if username == "" or password == "":
            messagebox.showerror("Error", "Please enter username and password.")
            return
        row = db.check_login(username, password)
        if row is None:
            messagebox.showerror("Login failed", "Wrong username or password entered.")
            return
        self.user = models.create_user(row)
        self.show_campus_select()

    def register(self):
        username = self.username_entry.get().strip()
        password = self.password_entry.get()
        if len(username) < 3 or len(password) < 4:
            messagebox.showerror("Error", "Username needs 3+ characters and password 4+ characters.")
            return
        if db.add_user(username, password, "lecturer"):
            messagebox.showinfo("Success", "Account created. You can now log in.")
        else:
            messagebox.showerror("Error", "That username is already taken.")

    # ================= CAMPUS SELECTION =================
    def show_campus_select(self):
        self.clear()
        frame = ttk.Frame(self.root, padding=30)
        frame.pack(expand=True)
        ttk.Label(frame, text=f"Welcome, {self.user.username}", font=("Arial", 14,
"bold")).pack(pady=10)
        ttk.Label(frame, text="Select a campus to work with:").pack()
        campuses = db.get_campuses()
        if self.user.role == "admin":      # admins can only use their own campus
            campuses = [c for c in campuses if c[0] == self.user.campus_id]
        self.campus_list = campuses
        self.campus_box = ttk.Combobox(frame, values=[c[1] for c in campuses], state="readonly")
        self.campus_box.current(0)
        self.campus_box.pack(pady=10)
        ttk.Button(frame, text="Continue", command=self.choose_campus).pack()

    def choose_campus(self):
        self.campus = self.campus_list[self.campus_box.current()]
        self.show_main()
```

```
    # ================== MAIN WINDOW ==================
    def show_main(self):
        self.clear()
        self.setup_colors(ROLE_COLORS[self.user.role])
        top = ttk.Frame(self.root, padding=8, style="Top.TFrame")
        top.pack(fill="x")
        ttk.Label(top, text=f"Active campus: {self.campus[1]}", style="Top.TLabel",
                  font=("Arial", 12, "bold")).pack(side="left")
        ttk.Label(top, text=f"     User: {self.user.username} ({self.user.role})",
                  style="Top.TLabel").pack(side="left")
        ttk.Button(top, text="Logout", command=self.show_login).pack(side="right")
        ttk.Button(top, text="Change campus", command=self.show_campus_select).pack(side="right",
padx=5)
        rule = models.get_rule(self.campus[1])
        ttk.Label(self.root, text="Booking policy in effect: " + rule.describe(),
                  foreground="blue", wraplength=860).pack(anchor="w", padx=8)

        self.tabs = ttk.Notebook(self.root)
        self.tabs.pack(fill="both", expand=True, padx=5, pady=5)
        self.tabs.bind("<<NotebookTabChanged>>", lambda e: self.refresh_all())

        if self.user.role == "lecturer":
            self.build_lecturer()
        elif self.user.role == "admin":
            self.build_admin()
        else:
            self.build_operator()
        self.refresh_all()

    def refresh_all(self):
        for func in self.refreshers:
            func()

    def new_tab(self, title):
        tab = ttk.Frame(self.tabs)
        self.tabs.add(tab, text=title)
        return tab

    # ================== LECTURER ==================
    def build_lecturer(self):
        # --- Tab 1: resources and availability ---
        tab = self.new_tab("Resources")
        self.res_tree = make_tree(tab, ("Code", "Name", "Availability"), (150, 300, 150))
        ttk.Button(tab, text="Refresh", command=self.refresh_all).pack(pady=3)
        self.refreshers.append(self.load_lecturer_resources)

        # --- Tab 2: make a booking ---
        tab = self.new_tab("Book a Resource")
        form = ttk.Frame(tab, padding=20)
        form.pack()
```

```
        ttk.Label(form, text="Resource:").grid(row=0, column=0, sticky="e", pady=5)
        self.res_box = ttk.Combobox(form, state="readonly", width=35)
        self.res_box.grid(row=0, column=1, pady=5)
        ttk.Label(form, text="Date (YYYY-MM-DD):").grid(row=1, column=0, sticky="e", pady=5)
        self.date_var = tk.StringVar(value=datetime.now().strftime("%Y-%m-%d"))
        ttk.Entry(form, textvariable=self.date_var, width=20).grid(row=1, column=1, sticky="w",
pady=5)
        ttk.Label(form, text="Start hour (0-23):").grid(row=2, column=0, sticky="e", pady=5)
        self.start_var = tk.StringVar(value="9")
        ttk.Spinbox(form, from_=0, to=23, textvariable=self.start_var, width=5).grid(row=2, column=1,
sticky="w", pady=5)
        ttk.Label(form, text="Duration (hours):").grid(row=3, column=0, sticky="e", pady=5)
        self.dur_var = tk.StringVar(value="1")
        ttk.Spinbox(form, from_=1, to=8, textvariable=self.dur_var, width=5).grid(row=3, column=1,
sticky="w", pady=5)
        ttk.Button(form, text="Book", command=self.make_booking).grid(row=4, column=0,
columnspan=2, pady=10)

        # --- Tab 3: my bookings ---
        tab = self.new_tab("My Bookings")
        self.my_tree = make_tree(tab, BOOKING_COLS, BOOKING_WIDTHS)
        ttk.Button(tab, text="Cancel selected (future bookings only)",
                   command=self.cancel_selected).pack(pady=3)
        self.refreshers.append(self.load_my_bookings)

    def load_lecturer_resources(self):
        self.resources = db.get_resources(self.campus[0])
        rows = []
        for r in self.resources:
            status = "In use now" if db.is_in_use_now(r[0]) else "Available"
            rows.append((r[0], r[1], r[2], status))
        fill_tree(self.res_tree, rows)
        self.res_box["values"] = [f"{r[1]} - {r[2]}" for r in self.resources]

    def load_my_bookings(self):
        fill_tree(self.my_tree, booking_rows(db.get_bookings(lecturer_id=self.user.user_id)))

    def make_booking(self):
        index = self.res_box.current()
        if index < 0:
            messagebox.showerror("Error", "Please select a resource.")
            return
        try:
            start = int(self.start_var.get())
            duration = int(self.dur_var.get())
        except ValueError:
            messagebox.showerror("Error", "Start hour and duration must be whole numbers.")
            return
        date_text = self.date_var.get().strip()
        error = models.validate_booking(self.campus[1], date_text, start, duration)
        if error:
```

```
            messagebox.showerror("Booking rule", error)
            return
        resource_id = self.resources[index][0]
        if db.has_overlap(resource_id, date_text, start, duration):
            messagebox.showerror("Not available", "That resource is already booked at this time.")
            return
        db.add_booking(resource_id, self.user.user_id, self.campus[0], date_text, start, duration)
        messagebox.showinfo("Success", "Booking created.")
        self.refresh_all()

    def cancel_selected(self):
        selected = self.my_tree.selection()
        if not selected:
            messagebox.showerror("Error", "Select a booking first.")
            return
        status = self.my_tree.item(selected[0], "values")[6]
        if status != "Upcoming":
            messagebox.showerror("Error", "Only upcoming bookings can be cancelled.")
            return
        if messagebox.askyesno("Confirm", "Cancel this booking?"):
            db.cancel_booking(int(selected[0]))
            self.refresh_all()

    # ================= CAMPUS ADMIN =================
    def build_admin(self):
        # --- Tab 1: manage resources ---
        tab = self.new_tab("Manage Resources")
        self.admin_tree = make_tree(tab, ("Code", "Name"), (200, 300))
        self.admin_tree.bind("<<TreeviewSelect>>", self.on_resource_select)
        form = ttk.Frame(tab)
        form.pack(pady=5)
        ttk.Label(form, text="Code:").grid(row=0, column=0)
        self.code_entry = ttk.Entry(form, width=15)
        self.code_entry.grid(row=0, column=1, padx=5)
        ttk.Label(form, text="Name:").grid(row=0, column=2)
        self.name_entry = ttk.Entry(form, width=25)
        self.name_entry.grid(row=0, column=3, padx=5)
        ttk.Button(form, text="Add", command=self.add_resource).grid(row=1, column=1, pady=5)
        ttk.Button(form, text="Update name", command=self.update_resource).grid(row=1, column=2)
        ttk.Button(form, text="Remove", command=self.remove_resource).grid(row=1, column=3)
        self.refreshers.append(self.load_admin_resources)

        # --- Tab 2: campus bookings ---
        tab = self.new_tab("Campus Bookings")
        self.campus_tree = make_tree(tab, BOOKING_COLS, BOOKING_WIDTHS)
        self.refreshers.append(self.load_campus_bookings)

        # --- Tab 3: report ---
        tab = self.new_tab("Campus Report")
        self.report_text = tk.Text(tab, height=20, state="disabled")
        self.report_text.pack(fill="both", expand=True, padx=5, pady=5)
```

```
        self.refreshers.append(self.load_campus_report)

    def load_admin_resources(self):
        rows = [(r[0], r[1], r[2]) for r in db.get_resources(self.campus[0])]
        fill_tree(self.admin_tree, rows)

    def load_campus_bookings(self):
        fill_tree(self.campus_tree, booking_rows(db.get_bookings(campus_id=self.campus[0])))

    def load_campus_report(self):
        report = db.campus_report(self.campus[0])
        text = f"REPORT FOR {self.campus[1].upper()}\n\n"
        text += f"Total bookings: {report['total']}\n"
        text += f"Average booking duration: {report['average']:.1f} hours\n\n"
        text += "Most frequently used resources:\n"
        for name, count in report["top"]:
            text += f"  {name}: {count} booking(s)\n"
        if not report["top"]:
            text += "  (no bookings yet)\n"
        text += "\nUsage trend (bookings per month):\n"
        for month, count in report["months"]:
            text += f"  {month}: {count}\n"
        self.report_text.config(state="normal")
        self.report_text.delete("1.0", "end")
        self.report_text.insert("1.0", text)
        self.report_text.config(state="disabled")

    def on_resource_select(self, event):
        selected = self.admin_tree.selection()
        if selected:
            code, name = self.admin_tree.item(selected[0], "values")
            self.code_entry.delete(0, "end")
            self.code_entry.insert(0, code)
            self.name_entry.delete(0, "end")
            self.name_entry.insert(0, name)

    def add_resource(self):
        code = self.code_entry.get().strip().upper()
        name = self.name_entry.get().strip()
        if code == "" or name == "":
            messagebox.showerror("Error", "Enter both a code and a name.")
            return
        if db.add_resource(code, name, self.campus[0]):
            self.refresh_all()
        else:
            messagebox.showerror("Duplicate", "A resource with that code already exists.")

    def update_resource(self):
        selected = self.admin_tree.selection()
        name = self.name_entry.get().strip()
        if not selected or name == "":
```

```
            messagebox.showerror("Error", "Select a resource and type the new name.")
            return
        db.update_resource(int(selected[0]), name)
        self.refresh_all()

    def remove_resource(self):
        selected = self.admin_tree.selection()
        if not selected:
            messagebox.showerror("Error", "Select a resource first.")
            return
        if db.has_future_bookings(int(selected[0])):
            messagebox.showerror("Error", "This resource still has active bookings.")
            return
        if messagebox.askyesno("Confirm", "Remove this resource?"):
            db.delete_resource(int(selected[0]))
            self.refresh_all()

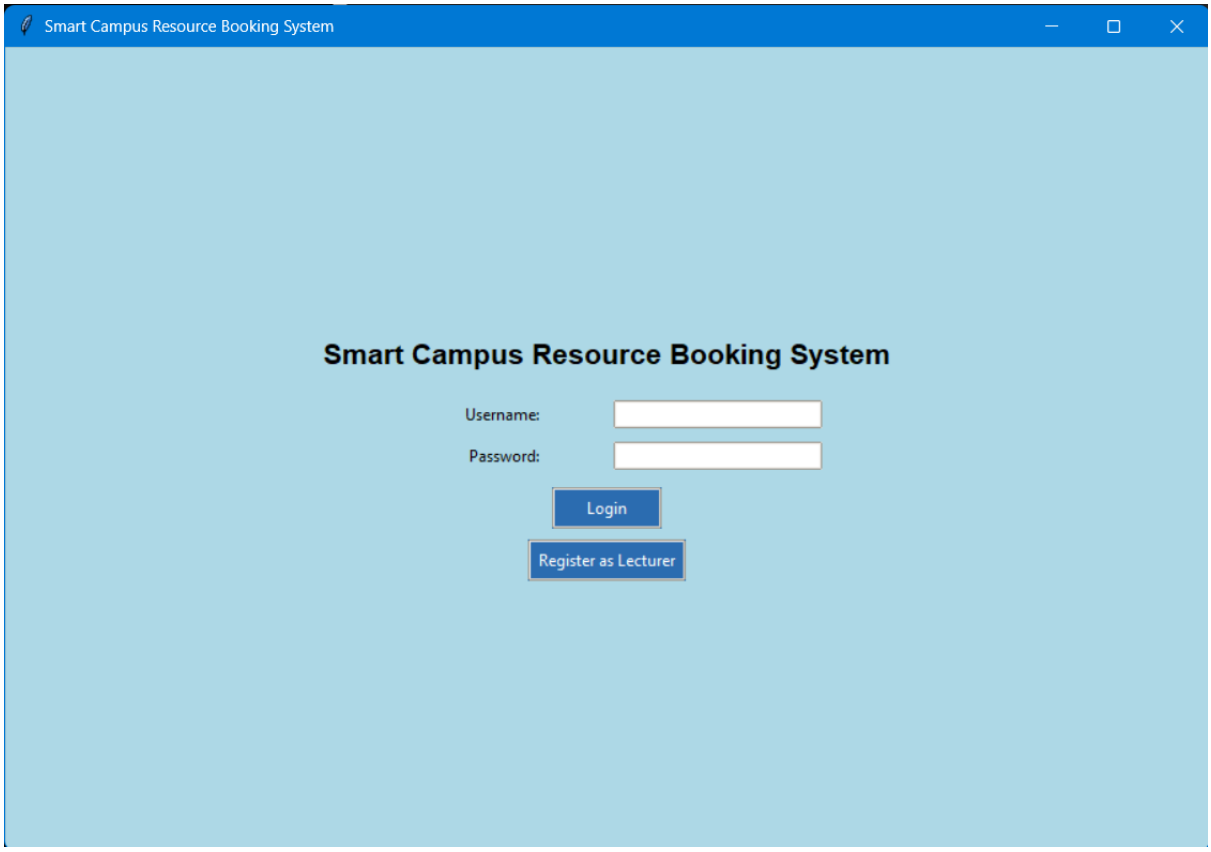
    # ================= SYSTEM OPERATOR ================
    def build_operator(self):
        tab = self.new_tab("All Bookings")
        self.all_tree = make_tree(tab, BOOKING_COLS, BOOKING_WIDTHS)
        self.refreshers.append(lambda: fill_tree(self.all_tree, booking_rows(db.get_bookings())))

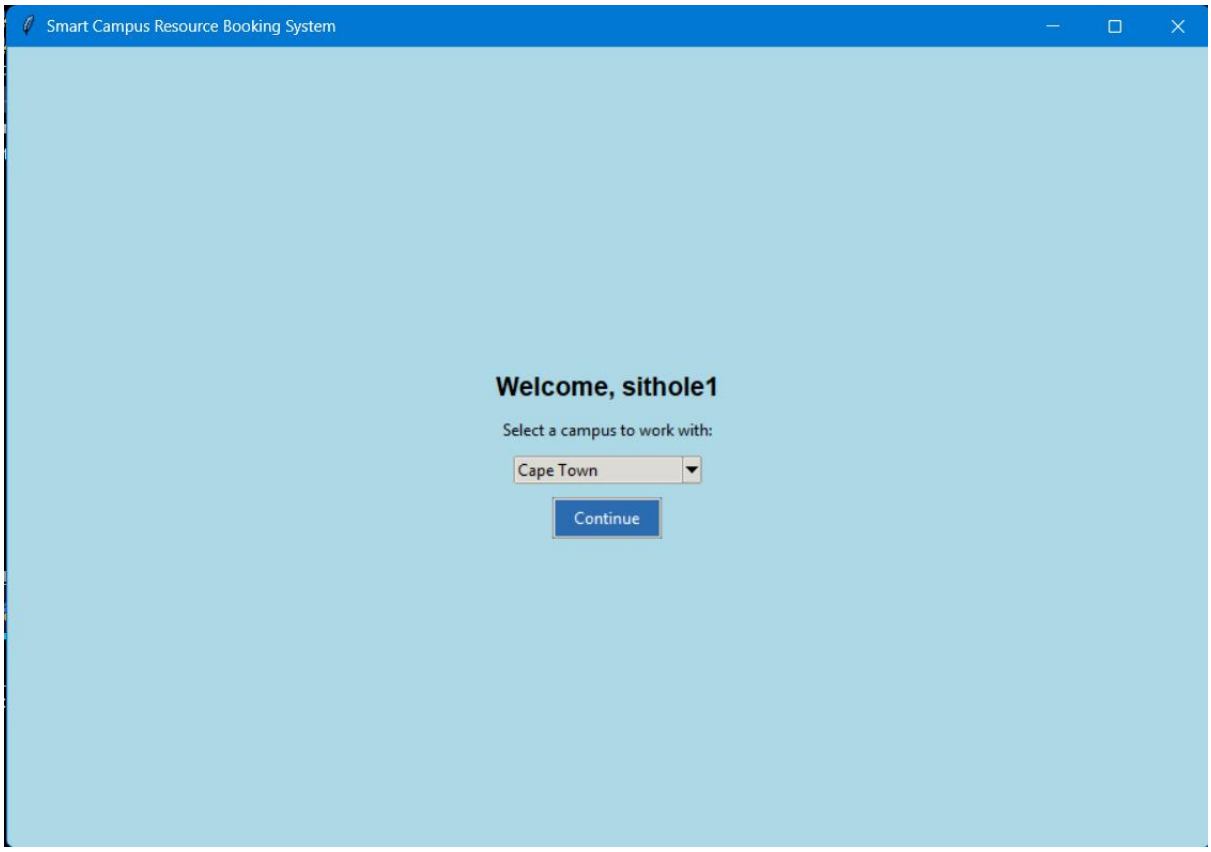
        tab = self.new_tab("All Resources")
        self.allres_tree = make_tree(tab, ("Campus", "Code", "Name"), (150, 150, 300))
        self.refreshers.append(lambda: fill_tree(self.allres_tree, db.get_all_resources()))

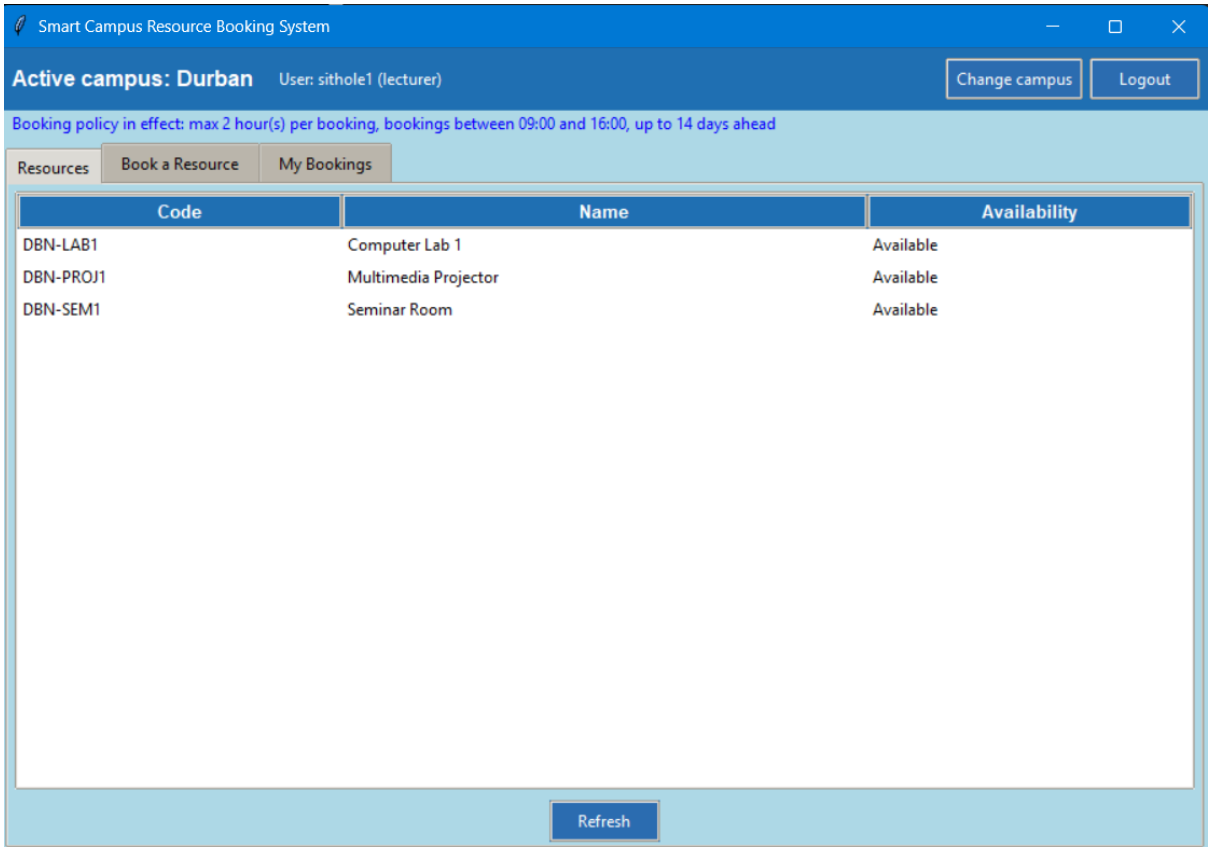
        tab = self.new_tab("Campus Comparison")
        self.compare_tree = make_tree(
            tab, ("Campus", "Total bookings", "Avg duration (h)", "Most used resource", "Max hours
rule"),
            (120, 110, 120, 200, 110))
        self.refreshers.append(self.load_comparison)

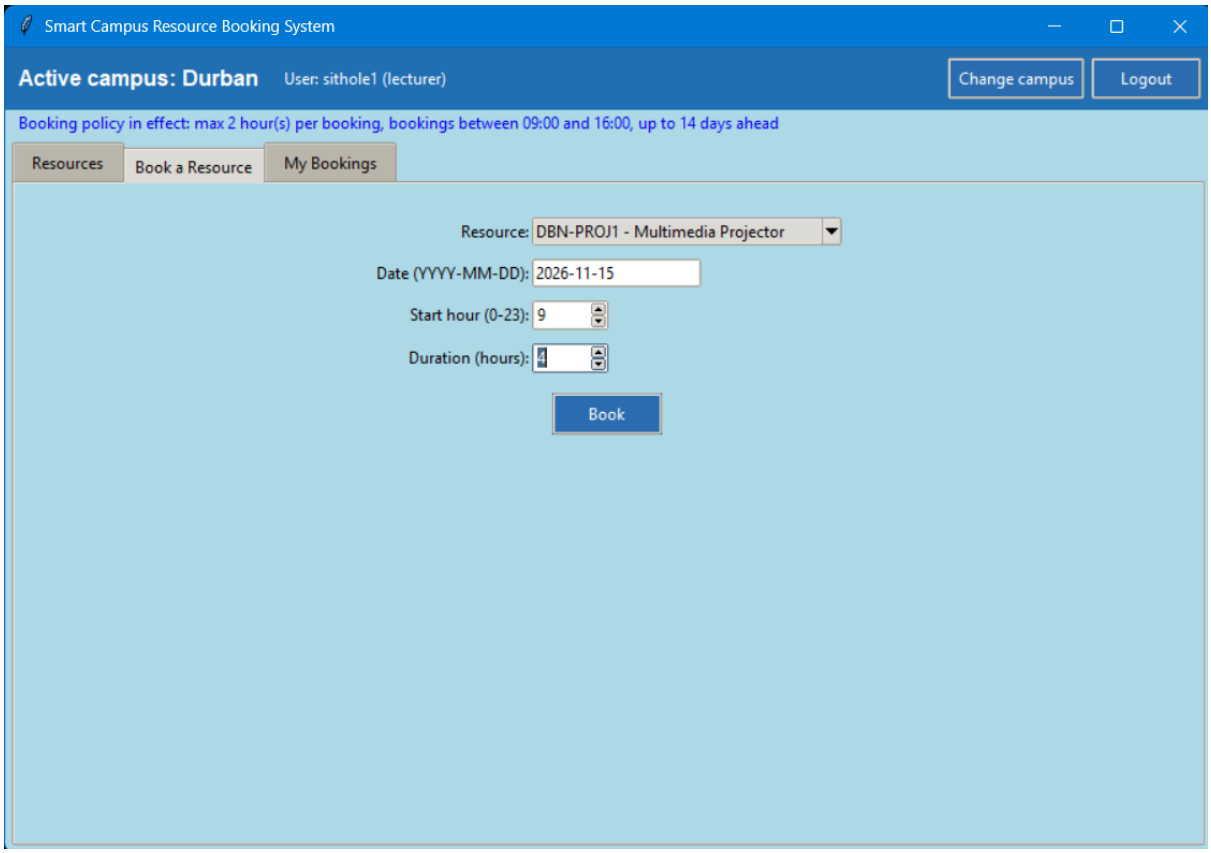
    def load_comparison(self):
        rows = []
        for campus_id, name in db.get_campuses():
            report = db.campus_report(campus_id)
            top = report["top"][0][0] if report["top"] else "-"
            rows.append((campus_id, name, report["total"], f"{report['average']:.1f}",
                         top, models.get_rule(name).max_hours))
        fill_tree(self.compare_tree, rows)

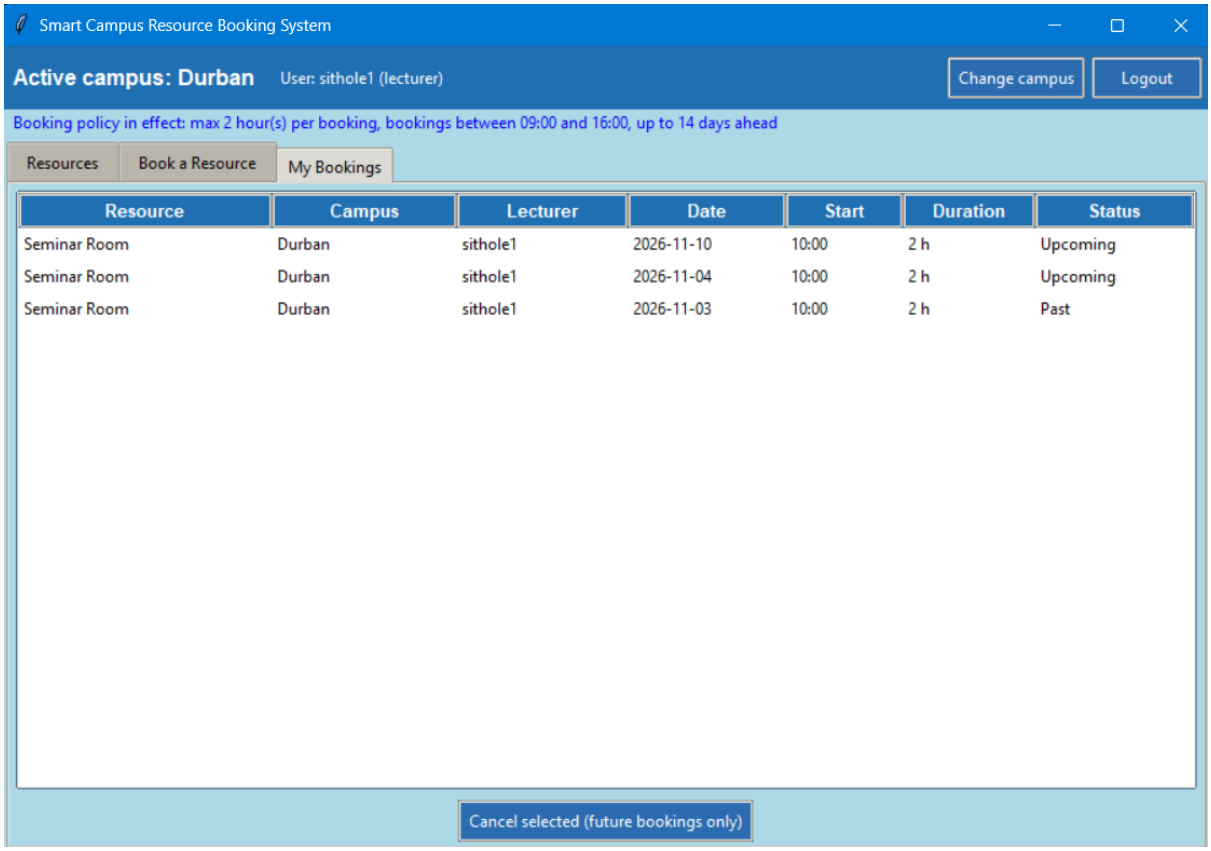

if __name__ == "__main__":
    db.create_tables()
    db.seed_data()
    window = tk.Tk()
    App(window)
    window.mainloop()
```

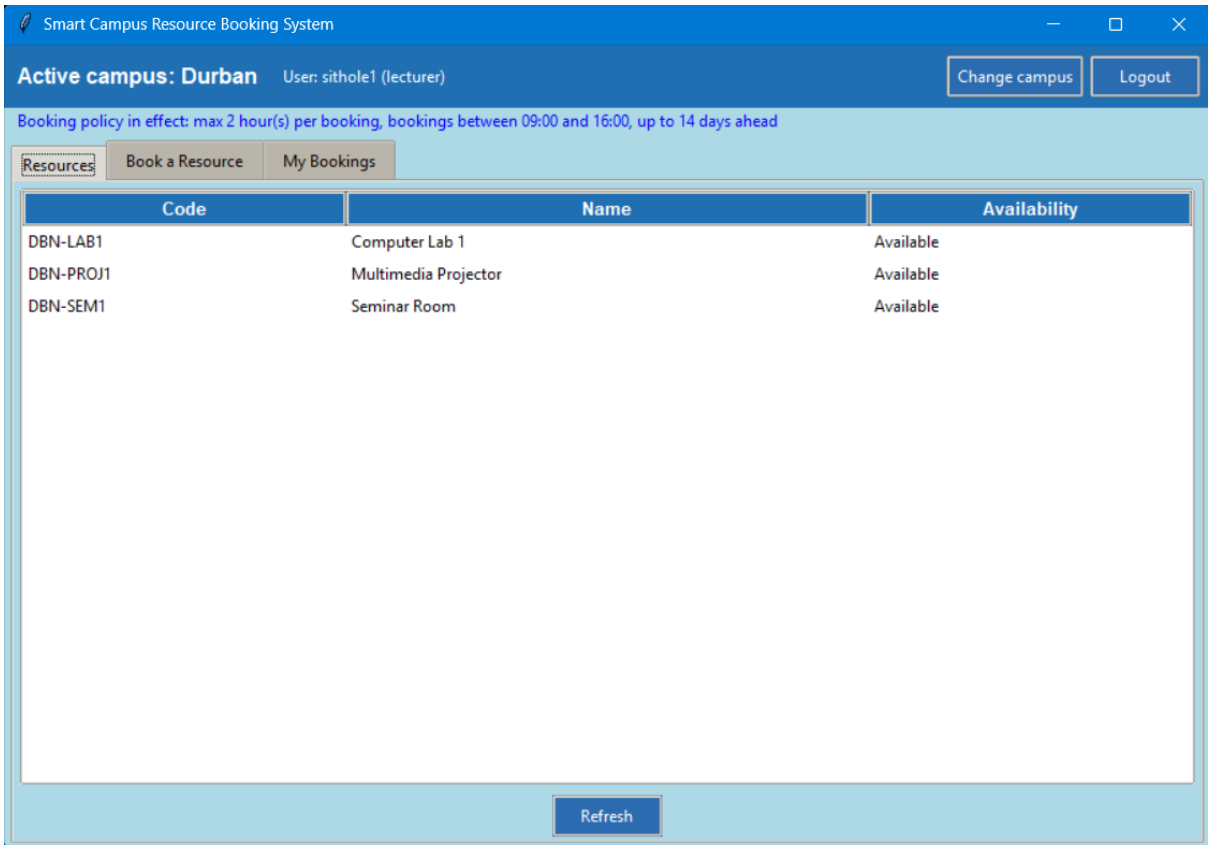

Smart Campus Resource Booking System
Smart Campus Resource Booking System
Username:
Password:
Login
Register as Lecturer

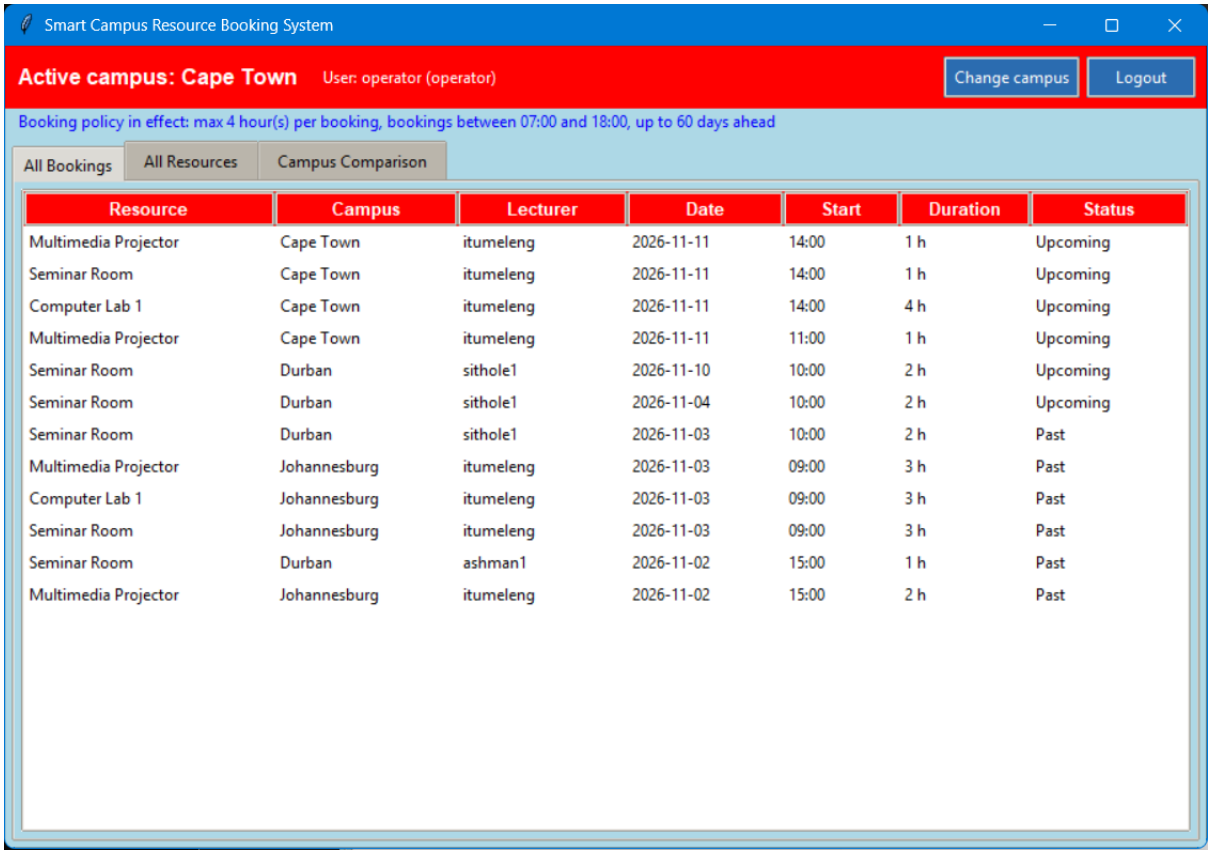

Smart Campus Resource Booking System
Welcome, sithole1
Select a campus to work with:
Cape Town
Continue

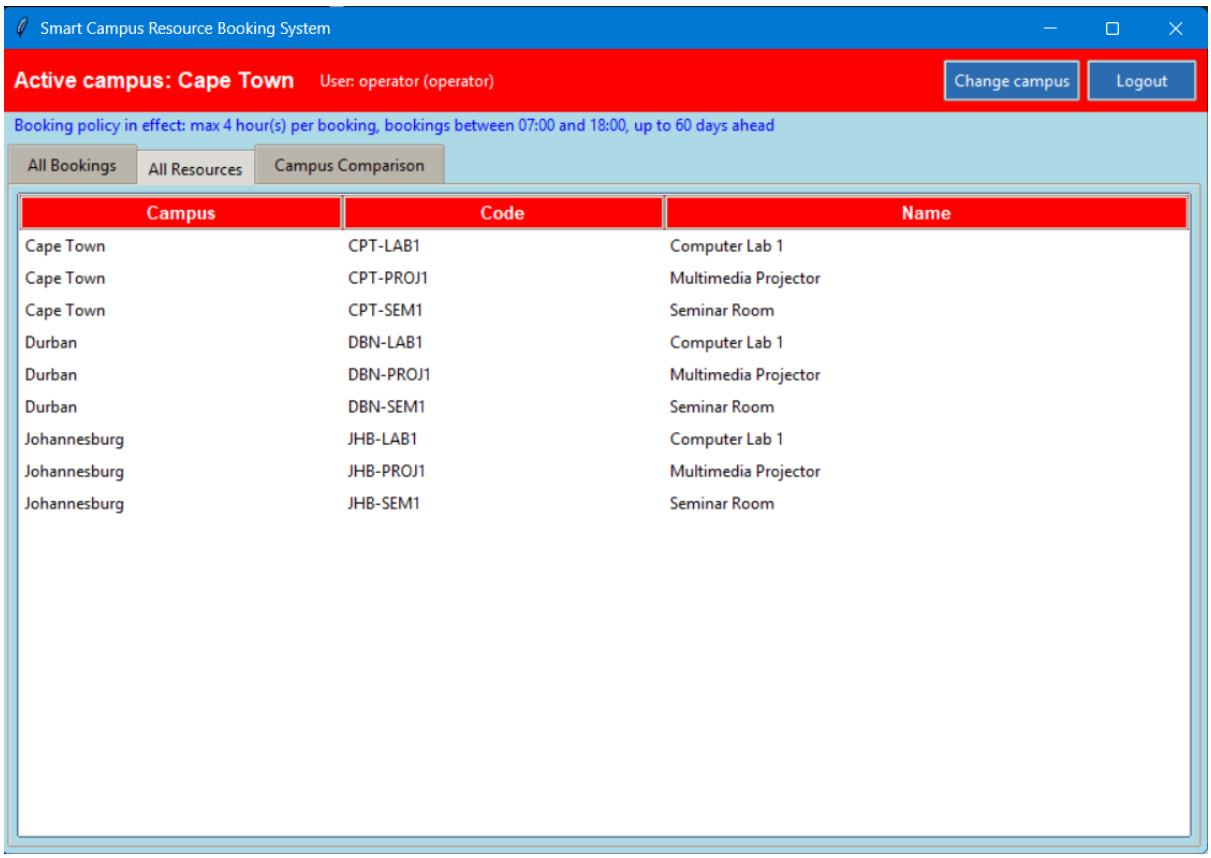

Smart Campus Resource Booking System
Active campus: Durban
User: sithole1 (lecturer)
Change campus
Logout
Booking policy in effect: max 2 hour(s) per booking, bookings between 09:00 and 16:00, up to 14 days ahead
Resources
Book a Resource
My Bookings
Code
Name
Availability
DBN-LAB1
Computer Lab 1
Available
DBN-PROJ1
Multimedia Projector
Available
DBN-SEM1
Seminar Room
Available
Refresh


Smart Campus Resource Booking System
Active campus: Durban
User: sithole1 (lecturer)
Change campus
Logout
Booking policy in effect: max 2 hour(s) per booking, bookings between 09:00 and 16:00, up to 14 days ahead
Resources
Book a Resource
My Bookings
Resource:
DBN-PROJ1 - Multimedia Projector
Date (YYYY-MM-DD):
2026-11-15
Start hour (0-23):
9
Duration (hours):
1
Book

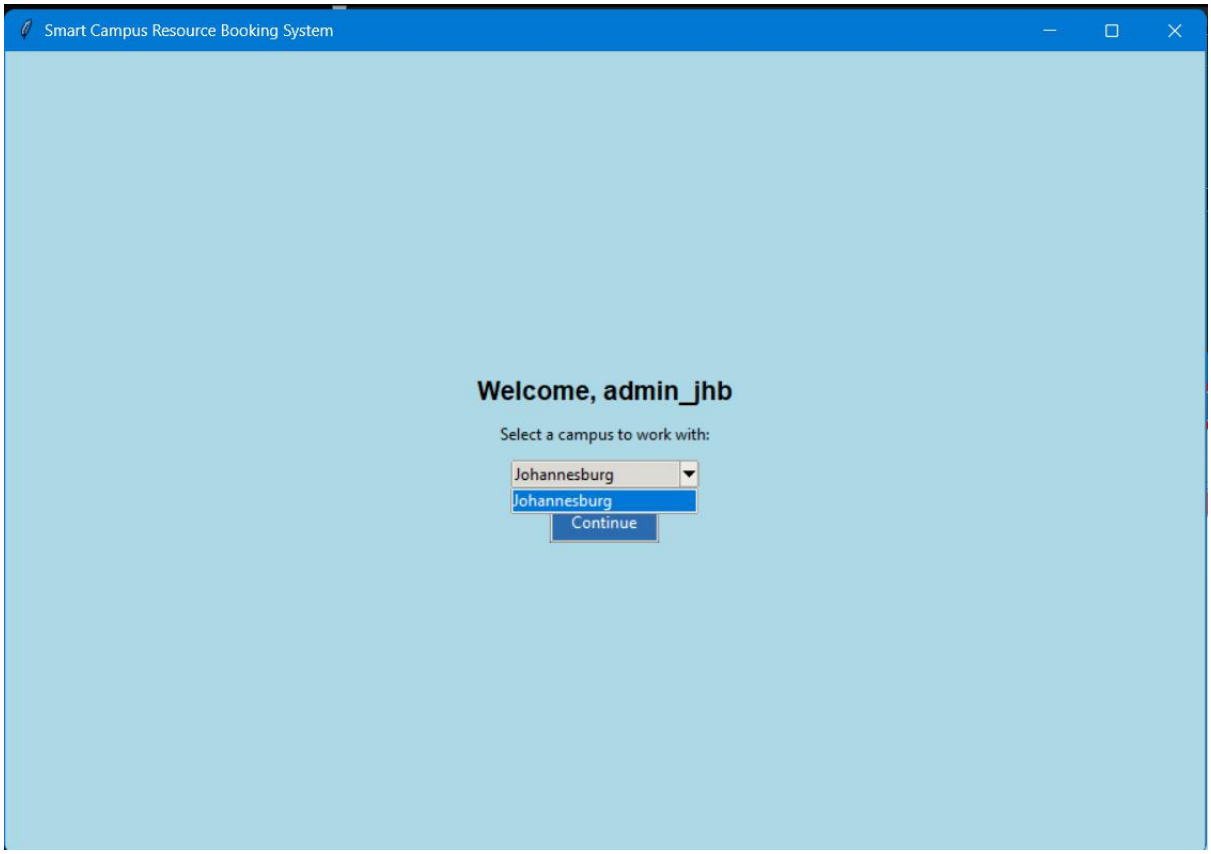

Smart Campus Resource Booking System
Active campus: Durban
User: sithole1 (lecturer)
Change campus
Logout
Booking policy in effect: max 2 hour(s) per booking, bookings between 09:00 and 16:00, up to 14 days ahead
Resources
Book a Resource
My Bookings
Resource
Campus
Lecturer
Date
Start
Duration
Status
Seminar Room
Durban
sithole1
2026-11-10
10:00
2 h
Upcoming
Seminar Room
Durban
sithole1
2026-11-04
10:00
2 h
Upcoming
Seminar Room
Durban
sithole1
2026-11-03
10:00
2 h
Past
Cancel selected (future bookings only)

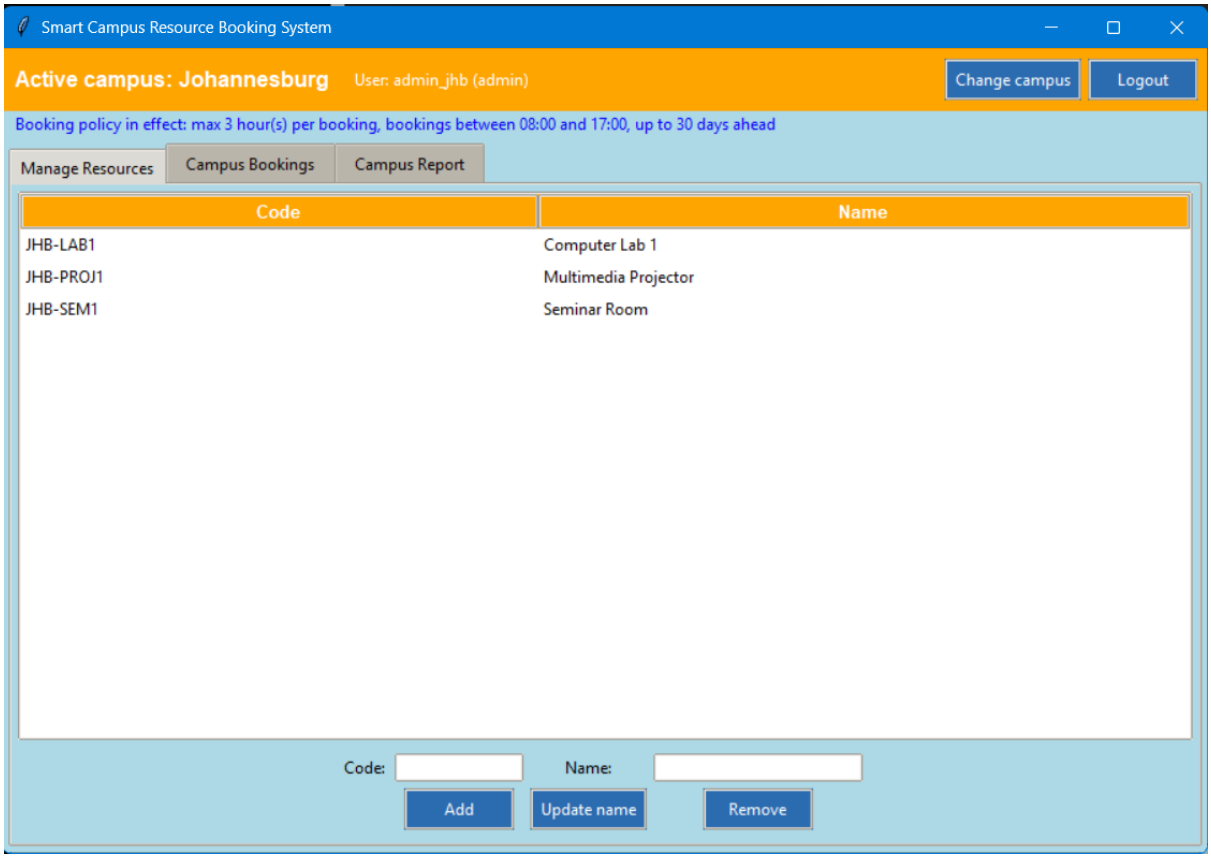

Smart Campus Resource Booking System
Active campus: Durban
User: sithole1 (lecturer)
Change campus
Logout
Booking policy in effect: max 2 hour(s) per booking, bookings between 09:00 and 16:00, up to 14 days ahead
Resources
Book a Resource
My Bookings
Code
Name
Availability
DBN-LAB1
Computer Lab 1
Available
DBN-PROJ1
Multimedia Projector
Available
DBN-SEM1
Seminar Room
Available
Refresh

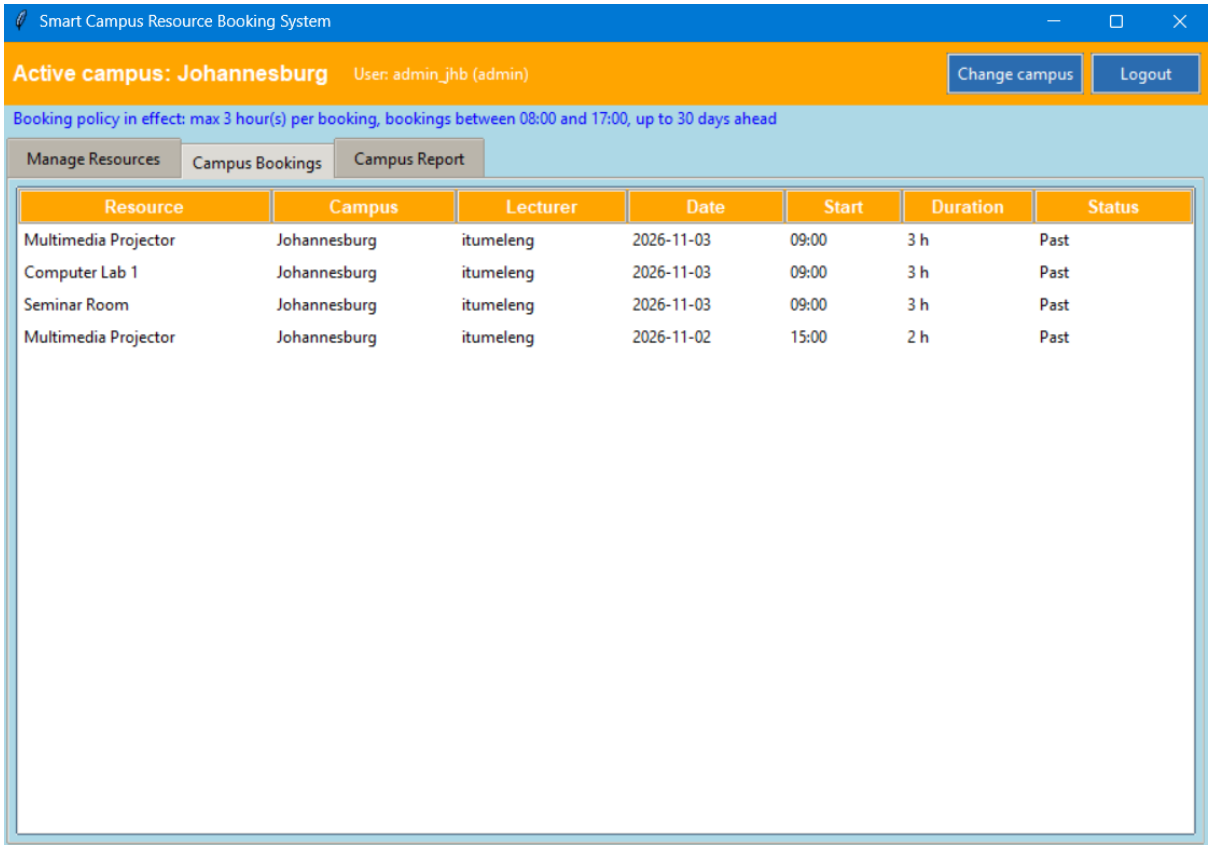

Smart Campus Resource Booking System
Active campus: Cape Town
User: operator (operator)
Change campus
Logout
Booking policy in effect: max 4 hour(s) per booking, bookings between 07:00 and 18:00, up to 60 days ahead
All Bookings
All Resources
Campus Comparison
Resource Campus Lecturer Date Start Duration Status
Multimedia Projector Cape Town itumeleng 2026-11-11 14:00 1 h Upcoming
Seminar Room Cape Town itumeleng 2026-11-11 14:00 1 h Upcoming
Computer Lab 1 Cape Town itumeleng 2026-11-11 14:00 4 h Upcoming
Multimedia Projector Cape Town itumeleng 2026-11-11 11:00 1 h Upcoming
Seminar Room Durban sithole1 2026-11-10 10:00 2 h Upcoming
Seminar Room Durban sithole1 2026-11-04 10:00 2 h Upcoming
Seminar Room Durban sithole1 2026-11-03 10:00 2 h Past
Multimedia Projector Johannesburg itumeleng 2026-11-03 09:00 3 h Past
Computer Lab 1 Johannesburg itumeleng 2026-11-03 09:00 3 h Past
Seminar Room Johannesburg itumeleng 2026-11-03 09:00 3 h Past
Seminar Room Durban ashman1 2026-11-02 15:00 1 h Past
Multimedia Projector Johannesburg itumeleng 2026-11-02 15:00 2 h Past

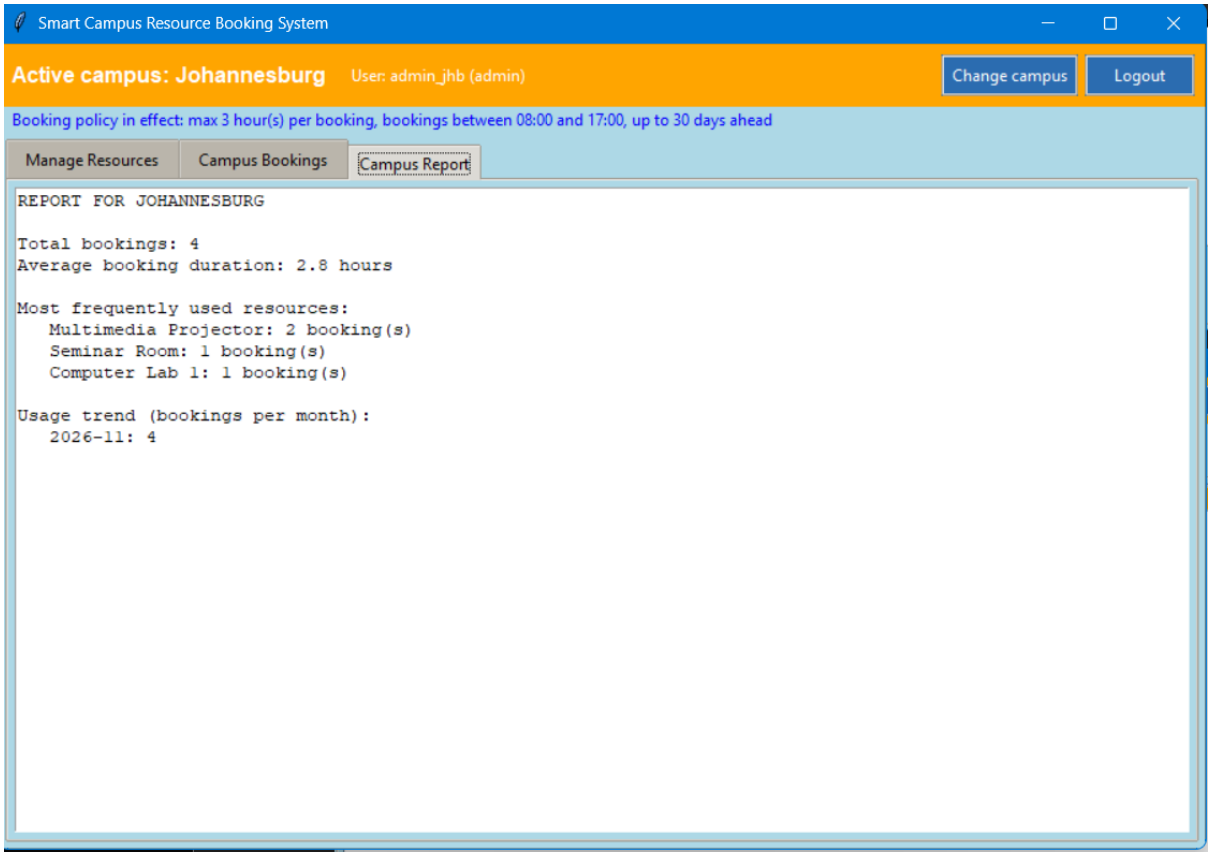

Smart Campus Resource Booking System
Active campus: Cape Town
User: operator (operator)
Change campus
Logout
Booking policy in effect: max 4 hour(s) per booking, bookings between 07:00 and 18:00, up to 60 days ahead
All Bookings
All Resources
Campus Comparison
Campus Code Name
Cape Town CPT-LAB1 Computer Lab 1
Cape Town CPT-PROJ1 Multimedia Projector
Cape Town CPT-SEM1 Seminar Room
Durban DBN-LAB1 Computer Lab 1
Durban DBN-PROJ1 Multimedia Projector
Durban DBN-SEM1 Seminar Room
Johannesburg JHB-LAB1 Computer Lab 1
Johannesburg JHB-PROJ1 Multimedia Projector
Johannesburg JHB-SEM1 Seminar Room

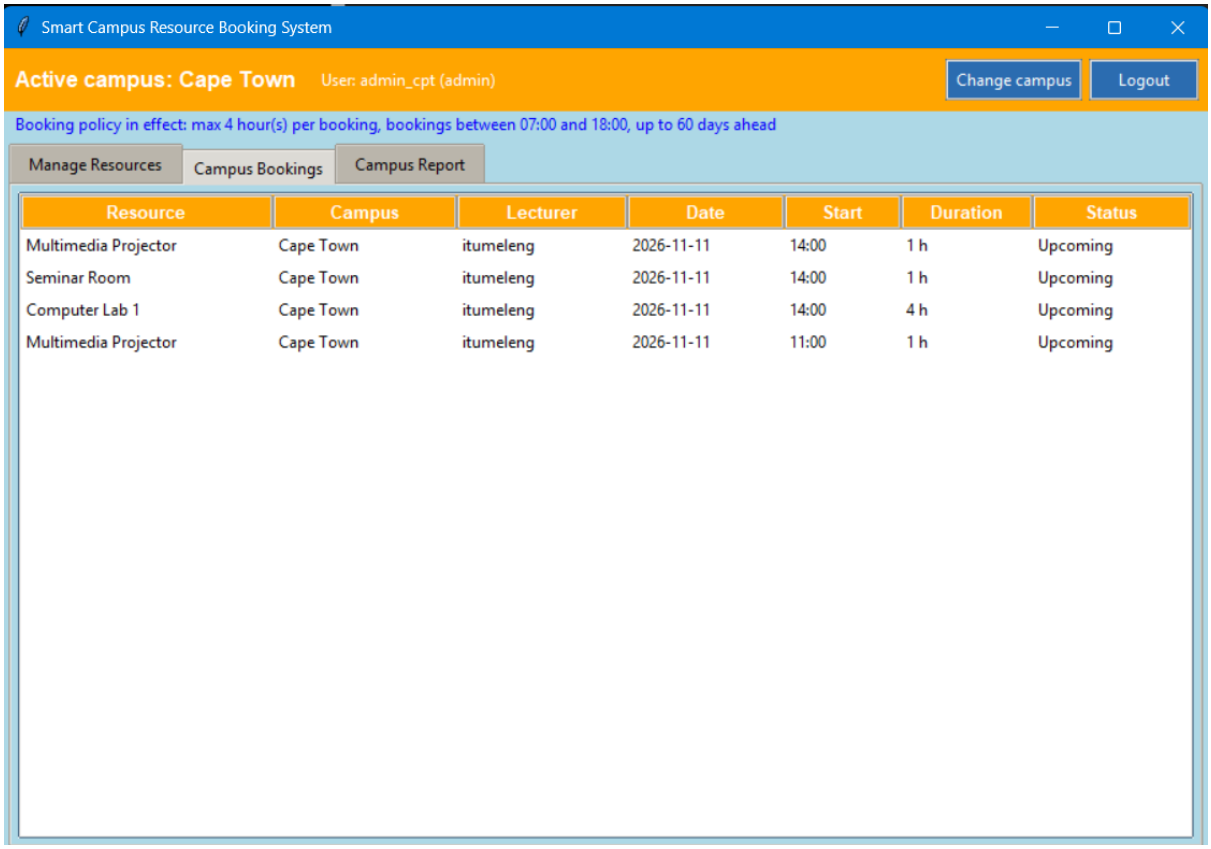

Smart Campus Resource Booking System
Active campus: Cape Town
User: operator (operator)
Change campus
Logout
Booking policy in effect: max 4 hour(s) per booking, bookings between 07:00 and 18:00, up to 60 days ahead
All Bookings
All Resources
Campus Comparison
Campus
Total bookings
Avg duration (h)
Most used resource
Max hours rule
Cape Town 4 1.8 Multimedia Projector 4
Durban 4 1.8 Seminar Room 2
Johannesburg 4 2.8 Multimedia Projector 3

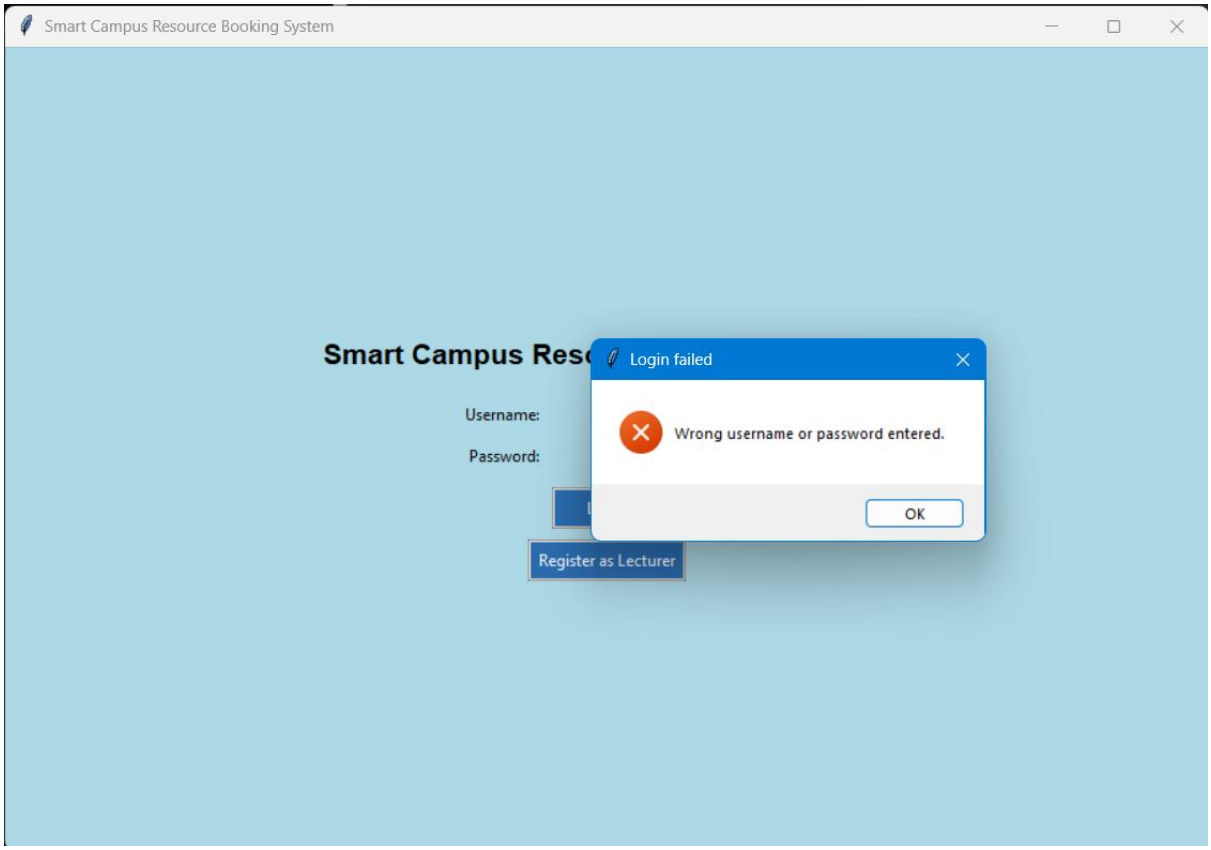

Smart Campus Resource Booking System
Welcome, admin_jhb
Select a campus to work with:
Johannesburg
Johannesburg
Continue


Smart Campus Resource Booking System
Active campus: Johannesburg
User: admin_jhb (admin)
Change campus
Logout
Booking policy in effect: max 3 hour(s) per booking, bookings between 08:00 and 17:00, up to 30 days ahead
Manage Resources
Campus Bookings
Campus Report
Code
Name
JHB-LAB1
Computer Lab 1
JHB-PROJ1
Multimedia Projector
JHB-SEM1
Seminar Room
Code:
Name:
Add
Update name
Remove


Smart Campus Resource Booking System
Active campus: Johannesburg
User: admin_jhb (admin)
Change campus
Logout
Booking policy in effect: max 3 hour(s) per booking, bookings between 08:00 and 17:00, up to 30 days ahead
Manage Resources
Campus Bookings
Campus Report
Resource
Campus
Lecturer
Date
Start
Duration
Status
Multimedia Projector
Johannesburg
itumeleng
2026-11-03
09:00
3 h
Past
Computer Lab 1
Johannesburg
itumeleng
2026-11-03
09:00
3 h
Past
Seminar Room
Johannesburg
itumeleng
2026-11-03
09:00
3 h
Past
Multimedia Projector
Johannesburg
itumeleng
2026-11-02
15:00
2 h
Past


Smart Campus Resource Booking System
Active campus: Johannesburg
User: admin_jhb (admin)
Change campus
Logout
Booking policy in effect: max 3 hour(s) per booking, bookings between 08:00 and 17:00, up to 30 days ahead
Manage Resources
Campus Bookings
Campus Report
REPORT FOR JOHANNESBURG
Total bookings: 4
Average booking duration: 2.8 hours
Most frequently used resources:
Multimedia Projector: 2 booking(s)
Seminar Room: 1 booking(s)
Computer Lab 1: 1 booking(s)
Usage trend (bookings per month):
2026-11: 4


Smart Campus Resource Booking System
Active campus: Cape Town
User: admin_cpt (admin)
Change campus
Logout
Booking policy in effect: max 4 hour(s) per booking, bookings between 07:00 and 18:00, up to 60 days ahead
Manage Resources
Campus Bookings
Campus Report
Resource
Campus
Lecturer
Date
Start
Duration
Status
Multimedia Projector
Cape Town
itumeleng
2026-11-11
14:00
1 h
Upcoming
Seminar Room
Cape Town
itumeleng
2026-11-11
14:00
1 h
Upcoming
Computer Lab 1
Cape Town
itumeleng
2026-11-11
14:00
4 h
Upcoming
Multimedia Projector
Cape Town
itumeleng
2026-11-11
11:00
1 h
Upcoming



Models.py codes

```
"""models.py - user classes (OOP) and campus booking rules."""
from datetime import datetime, timedelta


# ---------- Users: one base class + three child classes (inheritance) ----------
class User:
    def __init__(self, user_id, username, campus_id=None):
        # Underscore = "private" attributes (encapsulation); use the properties below
        self._user_id = user_id
        self._username = username
        self._campus_id = campus_id
        self.role = "user"
        self.permissions = []

    @property
    def user_id(self):
        return self._user_id

    @property
    def username(self):
        return self._username

    @property
```

```
    def campus_id(self):
        return self._campus_id

    def can(self, action):
        return action in self.permissions


class Lecturer(User):
    def __init__(self, user_id, username, campus_id=None):
        super().__init__(user_id, username, campus_id)
        self.role = "lecturer"
        self.permissions = ["book", "view_own", "cancel_own"]


class CampusAdmin(User):
    def __init__(self, user_id, username, campus_id=None):
        super().__init__(user_id, username, campus_id)
        self.role = "admin"
        self.permissions = ["manage_resources", "view_campus_bookings", "campus_report"]


class SystemOperator(User):
    def __init__(self, user_id, username, campus_id=None):
        super().__init__(user_id, username, campus_id)
        self.role = "operator"
        self.permissions = ["view_all", "compare_campuses"]


def create_user(row):
    """Turn a database row (id, username, role, campus_id) into the right class."""
    user_id, username, role, campus_id = row
    if role == "admin":
        return CampusAdmin(user_id, username, campus_id)
    if role == "operator":
        return SystemOperator(user_id, username, campus_id)
    return Lecturer(user_id, username, campus_id)


# ---------- Booking rules (kept separate from the GUI and database) ----------
class BookingRule:
    def __init__(self, max_hours, open_hour, close_hour, max_days_ahead):
        self.max_hours = max_hours
        self.open_hour = open_hour
        self.close_hour = close_hour
        self.max_days_ahead = max_days_ahead

    def describe(self):
        return (f"max {self.max_hours} hour(s) per booking, bookings between "
```

```
            f"{self.open_hour:02d}:00 and {self.close_hour:02d}:00, "
            f"up to {self.max_days_ahead} days ahead")

    def check(self, start, duration, days_ahead):
        """Return an error message, or '' if the booking follows the rules."""
        if duration > self.max_hours:
            return f"This campus allows a maximum of {self.max_hours} hour(s) per booking."
        if start < self.open_hour or start + duration > self.close_hour:
            return f"Bookings must be between {self.open_hour:02d}:00 and {self.close_hour:02d}:00."
        if days_ahead > self.max_days_ahead:
            return f"You can only book up to {self.max_days_ahead} days ahead."
        return ""


# To add a new campus rule, just add one more line to this dictionary.
RULES = {
    "Johannesburg": BookingRule(3, 8, 17, 30),
    "Durban": BookingRule(2, 9, 16, 14),
    "Cape Town": BookingRule(4, 7, 18, 60),
}
DEFAULT_RULE = BookingRule(2, 8, 17, 14)


def get_rule(campus_name):
    return RULES.get(campus_name, DEFAULT_RULE)


def validate_booking(campus_name, date_text, start, duration):
    """Check date format, not in the past, and the campus rules. Returns error text or ''."""
    try:
        booking_date = datetime.strptime(date_text, "%Y-%m-%d")
    except ValueError:
        return "Date must be in the format YYYY-MM-DD."
    if duration < 1:
        return "Duration must be at least 1 hour."
    if start < 0 or start > 23:
        return "Start hour must be between 0 and 23."
    if booking_date + timedelta(hours=start) <= datetime.now():
        return "You cannot book a time in the past."
    days_ahead = (booking_date.date() - datetime.now().date()).days
    return get_rule(campus_name).check(start, duration, days_ahead)


def booking_state(date_text, start, status):
    """Label a booking as Cancelled, Upcoming or Past."""
    if status == "cancelled":
        return "Cancelled"
```

```
when = datetime.strptime(date_text, "%Y-%m-%d") + timedelta(hours=start)
return "Upcoming" if when > datetime.now() else "Past"
```

Database.py codes

```
"""database.py - all SQLite code lives here (tables, queries, reports)."""
import sqlite3
import hashlib
from datetime import datetime

DB_NAME = "campus.db"


def connect():
    return sqlite3.connect(DB_NAME)


def hash_password(password):
    """Store a hash of the password, never the plain text."""
    return hashlib.sha256(password.encode()).hexdigest()


def create_tables():
    conn = connect()
    c = conn.cursor()
    c.execute("""CREATE TABLE IF NOT EXISTS campuses (
                 id INTEGER PRIMARY KEY AUTOINCREMENT,
                 name TEXT UNIQUE NOT NULL)""")
    c.execute("""CREATE TABLE IF NOT EXISTS users (
                 id INTEGER PRIMARY KEY AUTOINCREMENT,
                 username TEXT UNIQUE NOT NULL,
                 password TEXT NOT NULL,
                 role TEXT NOT NULL,
                 campus_id INTEGER)""")
    # UNIQUE on code stops duplicate resources
    c.execute("""CREATE TABLE IF NOT EXISTS resources (
                 id INTEGER PRIMARY KEY AUTOINCREMENT,
                 code TEXT UNIQUE NOT NULL,
                 name TEXT NOT NULL,
                 campus_id INTEGER NOT NULL)""")
    c.execute("""CREATE TABLE IF NOT EXISTS bookings (
                 id INTEGER PRIMARY KEY AUTOINCREMENT,
                 resource_id INTEGER NOT NULL,
                 lecturer_id INTEGER NOT NULL,
                 campus_id INTEGER NOT NULL,
                 date TEXT NOT NULL,
                 start_hour INTEGER NOT NULL,
                 duration INTEGER NOT NULL,
```

```
            status TEXT NOT NULL DEFAULT 'active')""")
    conn.commit()
    conn.close()


def seed_data():
    """Add starting campuses, admins, an operator and some resources (only once)."""
    conn = connect()
    c = conn.cursor()
    if c.execute("SELECT COUNT(*) FROM campuses").fetchone()[0] > 0:
        conn.close()
        return
    campuses = [("Johannesburg", "jhb"), ("Durban", "dbn"), ("Cape Town", "cpt")]
    for name, short in campuses:
        c.execute("INSERT INTO campuses (name) VALUES (?)", (name,))
        campus_id = c.lastrowid
        c.execute("INSERT INTO users (username, password, role, campus_id) VALUES (?,?,?,?)",
                  ("admin_" + short, hash_password("admin123"), "admin", campus_id))
        c.execute("INSERT INTO resources (code, name, campus_id) VALUES (?,?,?)",
                  (short.upper() + "-LAB1", "Computer Lab 1", campus_id))
        c.execute("INSERT INTO resources (code, name, campus_id) VALUES (?,?,?)",
                  (short.upper() + "-PROJ1", "Multimedia Projector", campus_id))
        c.execute("INSERT INTO resources (code, name, campus_id) VALUES (?,?,?)",
                  (short.upper() + "-SEM1", "Seminar Room", campus_id))
    c.execute("INSERT INTO users (username, password, role, campus_id) VALUES (?,?,?,?)",
              ("operator", hash_password("oper123"), "operator", None))
    conn.commit()
    conn.close()


# ---------- users ----------
def add_user(username, password, role, campus_id=None):
    """Returns True if saved, False if the username already exists."""
    conn = connect()
    try:
        conn.execute("INSERT INTO users (username, password, role, campus_id) VALUES
(?,?,?,?)",
                     (username, hash_password(password), role, campus_id))
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False
    finally:
        conn.close()


def check_login(username, password):
    """Returns (id, username, role, campus_id) or None."""
```

```
    conn = connect()
    row = conn.execute("SELECT id, username, role, campus_id FROM users "
                       "WHERE username=? AND password=?",
                       (username, hash_password(password))).fetchone()
    conn.close()
    return row


# ---------- campuses ----------
def get_campuses():
    conn = connect()
    rows = conn.execute("SELECT id, name FROM campuses ORDER BY name").fetchall()
    conn.close()
    return rows


# ---------- resources ----------
def get_resources(campus_id):
    conn = connect()
    rows = conn.execute("SELECT id, code, name FROM resources WHERE campus_id=? ORDER
BY code",
                        (campus_id,)).fetchall()
    conn.close()
    return rows


def get_all_resources():
    conn = connect()
    rows = conn.execute("SELECT r.id, c.name, r.code, r.name FROM resources r "
                        "JOIN campuses c ON r.campus_id = c.id ORDER BY c.name, r.code").fetchall()
    conn.close()
    return rows


def add_resource(code, name, campus_id):
    """Returns False if a resource with this code already exists."""
    conn = connect()
    try:
        conn.execute("INSERT INTO resources (code, name, campus_id) VALUES (?,?,?)",
                     (code, name, campus_id))
        conn.commit()
        return True
    except sqlite3.IntegrityError:
        return False
    finally:
        conn.close()
```

```
def update_resource(resource_id, new_name):
    conn = connect()
    conn.execute("UPDATE resources SET name=? WHERE id=?", (new_name, resource_id))
    conn.commit()
    conn.close()


def has_future_bookings(resource_id):
    today = datetime.now().strftime("%Y-%m-%d")
    conn = connect()
    count = conn.execute("SELECT COUNT(*) FROM bookings WHERE resource_id=? "
                         "AND status='active' AND date>=?", (resource_id, today)).fetchone()[0]
    conn.close()
    return count > 0


def delete_resource(resource_id):
    conn = connect()
    conn.execute("DELETE FROM resources WHERE id=?", (resource_id,))
    conn.commit()
    conn.close()


def is_in_use_now(resource_id):
    now = datetime.now()
    conn = connect()
    count = conn.execute("SELECT COUNT(*) FROM bookings WHERE resource_id=? AND
status='active' "
                         "AND date=? AND start_hour<=? AND start_hour+duration>?",
                         (resource_id, now.strftime("%Y-%m-%d"), now.hour, now.hour)).fetchone()[0]
    conn.close()
    return count > 0


# ---------- bookings ----------
def has_overlap(resource_id, date, start, duration):
    """True if an active booking for this resource overlaps the requested time."""
    end = start + duration
    conn = connect()
    count = conn.execute("SELECT COUNT(*) FROM bookings WHERE resource_id=? AND date=?
"
                         "AND status='active' AND start_hour<? AND start_hour+duration>?",
                         (resource_id, date, end, start)).fetchone()[0]
    conn.close()
    return count > 0


def add_booking(resource_id, lecturer_id, campus_id, date, start, duration):
```

```
    conn = connect()
    conn.execute("INSERT INTO bookings (resource_id, lecturer_id, campus_id, date, start_hour,
duration) "
                 "VALUES (?,?,?,?,?,?)", (resource_id, lecturer_id, campus_id, date, start, duration))
    conn.commit()
    conn.close()


def cancel_booking(booking_id):
    conn = connect()
    conn.execute("UPDATE bookings SET status='cancelled' WHERE id=?", (booking_id,))
    conn.commit()
    conn.close()


def get_bookings(campus_id=None, lecturer_id=None):
    """Rows: (id, resource, campus, lecturer, date, start, duration, status)."""
    sql = ("SELECT b.id, r.name, c.name, u.username, b.date, b.start_hour, b.duration, b.status "
           "FROM bookings b JOIN resources r ON b.resource_id=r.id "
           "JOIN campuses c ON b.campus_id=c.id JOIN users u ON b.lecturer_id=u.id")
    params = []
    if campus_id is not None:
        sql += " WHERE b.campus_id=?"
        params.append(campus_id)
    elif lecturer_id is not None:
        sql += " WHERE b.lecturer_id=?"
        params.append(lecturer_id)
    sql += " ORDER BY b.date DESC, b.start_hour DESC"
    conn = connect()
    rows = conn.execute(sql, params).fetchall()
    conn.close()
    return rows


# ---------- reports ----------
def campus_report(campus_id):
    """Returns a dictionary with the report figures for one campus."""
    conn = connect()
    total = conn.execute("SELECT COUNT(*) FROM bookings WHERE campus_id=? AND
status='active'",
                         (campus_id,)).fetchone()[0]
    avg = conn.execute("SELECT AVG(duration) FROM bookings WHERE campus_id=? AND
status='active'",
                       (campus_id,)).fetchone()[0]
    top = conn.execute("SELECT r.name, COUNT(*) AS n FROM bookings b JOIN resources r "
                       "ON b.resource_id=r.id WHERE b.campus_id=? AND b.status='active' "
                       "GROUP BY r.name ORDER BY n DESC LIMIT 3", (campus_id,)).fetchall()
    months = conn.execute("SELECT substr(date,1,7), COUNT(*) FROM bookings WHERE
```

```
campus_id=? "
                "AND status='active' GROUP BY substr(date,1,7) ORDER BY 1",
                (campus_id,)).fetchall()
  conn.close()
  return {"total": total, "average": avg or 0, "top": top, "months": months}
```

Readme.md

Smart Campus Resource Booking System
Run: python main.py (Python 3, no extra libraries needed)

https://github.com/kingmonty202-gif/Programming-512-Assignment-final.git

Test accounts:
 Campus admins : admin_jhb / admin_dbn / admin_cpt password: admin123
 System operator: operator password: oper123
 lecturer:sithole1 password:123456
 lecturer:itumeleng password:itu123
 lecturer:ashman password:123456
 Lecturer : click "Register as Lecturer" on the login screen and create lecturer account, once account is created click login to access all available resources and make booking of choice

Features

- Three user roles: Lecturer, Campus Admin, System Operator
- Role-based dashboards built with Tkinter
- SQLite database for persistence
- Campus-specific booking policies (applied automatically)
- Overlap detection and duration limits
- Booking history and cancellation
- Campus-level reports and cross-campus comparison

Conclusion

This assignment produced a working Smart Campus Resource Booking System that meets the main requirements of the case study. Lecturers can register, log in, select a campus, book resources and manage their bookings. Campus administrators can add, update and remove resources and view bookings and usage reports for their campus. System operators can view data across all campuses and compare their usage. The system prevents overlapping bookings, rejects duplicate resources and applies a different booking policy for each campus. Because all information is saved in an SQLite database, it remains available between program runs.

Building the system gave practical experience in using classes and inheritance to model user roles, dictionaries and lists to organise data, and SQL to store and retrieve information. It also showed the importance of input validation and error handling, because users can enter incorrect dates, times or durations. Keeping the booking rules in separate classes made the code easier to read and means a new campus can be added with very little change to the existing code.

The system has some limitations. Bookings are made in whole hours, priority access rules are not implemented, and the interface is basic. Future improvements could include email notifications, a calendar view, booking approval for high-demand resources, more detailed reports and charts, and stronger password security. Overall, the system shows how a centralised digital solution can reduce booking conflicts, improve accountability and give management better information to support decisions.

References

Lutz, M. (2013) *Learning Python*. 5th edn. Sebastopol: O'Reilly Media.

Python Software Foundation (2026a) *hashlib — Secure hashes and message digests*. Available at: https://docs.python.org/3/library/hashlib.html (Accessed: 25 October 2026).

Python Software Foundation (2026b) *sqlite3 — DB-API 2.0 interface for SQLite databases*. Available at: https://docs.python.org/3/library/sqlite3.html (Accessed: 25 October 2026).

Python Software Foundation (2026c) *tkinter — Python interface to Tcl/Tk*. Available at: https://docs.python.org/3/library/tkinter.html (Accessed: 25 October 2026).

Sommerville, I. (2016) *Software engineering*. 10th edn. Harlow: Pearson.

SQLite (n.d.) *About SQLite*. Available at: https://www.sqlite.org/about.html (Accessed: 25 October 2026).